

כמה ילדים יכולים לשחק בו זמנית

נמדד ב-10 בספטמבר 2026 · הנתונים מ-tools/loadtest.py

0.14/s

בקשות לכל ילד

34 קריאות במשחק של 4 דקות

~10/s

בקשות לטיטור

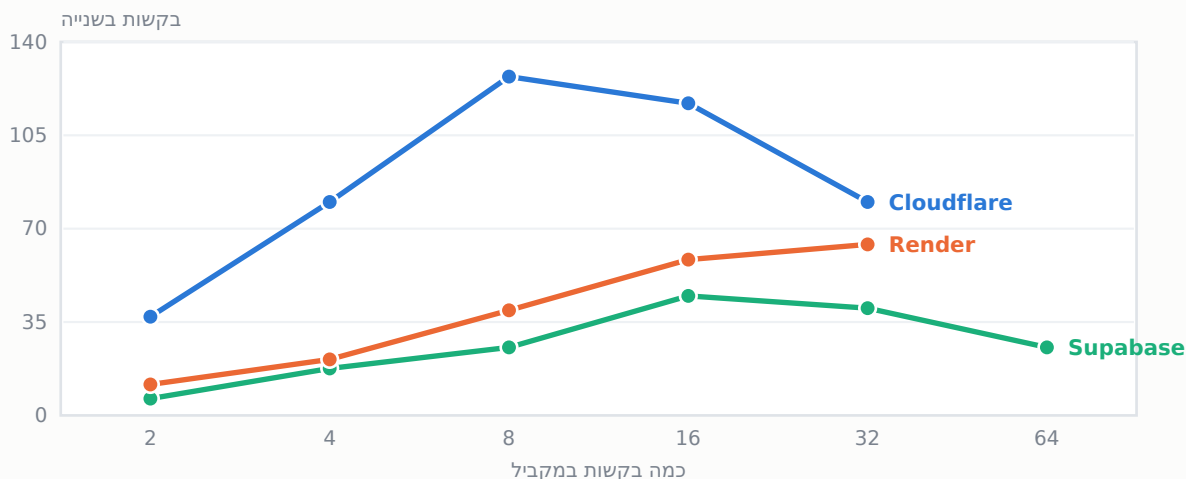
מוגבל על ידי קריאת המודל

~600

ילדים משחקים בו זמנית

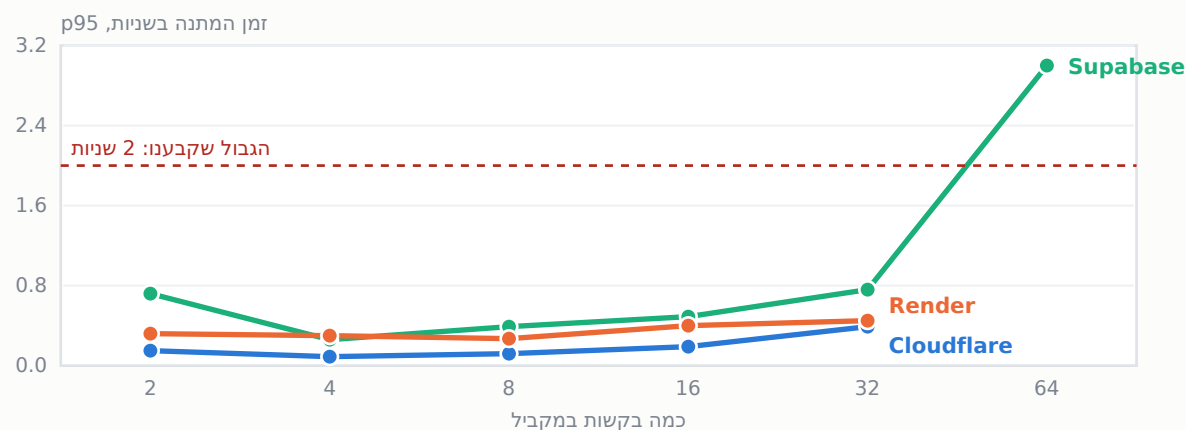
מוגבל על ידי Supabase

מה שנמדד: כמה נטענים בשנייה, לפי כמה בו זמנית



שלושת הקווים מגיעים לרמה ואז יורדים. הירידה של Cloudflare ו-Render היא מכונת המדידה, שהיא בעלת שני מעבדים בלבד וה-CPU שלה ירד ל-27% פנוי. Supabase הוא היחיד שנצפתה בו האטה אמיתית.

אותו ניסוי, אבל כמה זמן ילד מחכה



ב-64 בקשות במקביל, Supabase חוצה שתי שניות. שם עובר הגבול, ובלי שגיאה אחת — הבקשות פשוט ממתינות בתור. ילד רואה מסך שנעצר, לא הודעת שגיאה.

איך מגיעים מזה ל-600

משחק של עשר שאלות עושה 34 קריאות ל-Supabase לאורך כארבע דקות, כלומר 0.14 בשנייה לילד. עמד ב-90 בקשות בשנייה בלי שגיאות, ו-90 חלקי 0.14 הוא כ-600 ילדים במקביל Supabase וזו רצפה, לא תקרה: המדידה נחסמה על ידי מכונת המדידה. הכלי מוכן להרצה ממכונה גדולה יותר.

הנקודה החשובה קודם: גודל השרת לא משפיע על המשחקים בכלל. המשחקים אינם נוגעים ב-Render. הם קבצים סטטיים מ-Cloudflare, ובסיס נתונים ב-Supabase. שרת גדול יותר לא יוסיף אפילו שחקן אחד. רק שדרוג של Supabase יוסיף.

הטיטור: מה כל שדרוג קונה

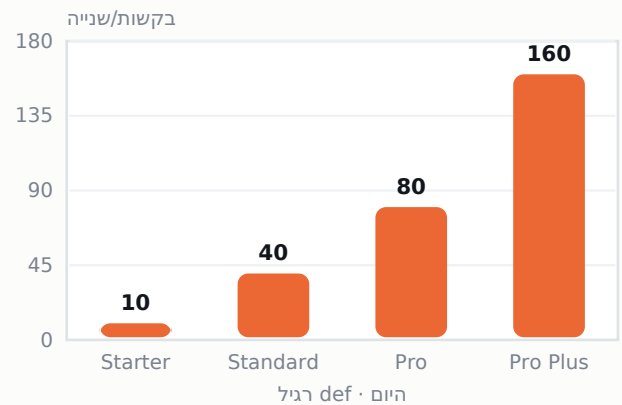
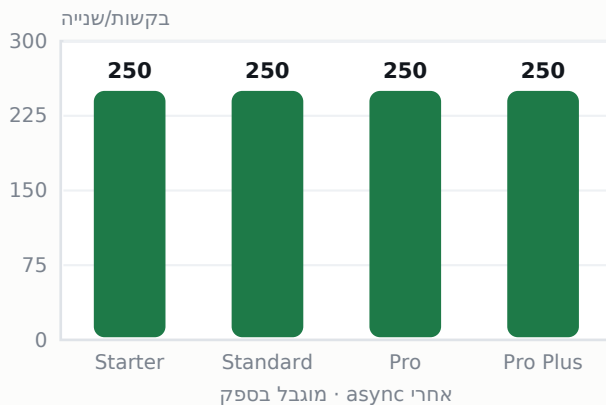
כל 19 התיבים בטיטור העברי מוגדרים כ-def רגיל ולא async. FastAPI מריץ כזה בבריכה של 40 חוטים, וכל בקשה תופסת חוט לכל אורך קריאת המודל — שניות, לא אלפיות.

תוכנית Render	משאבים	עובדים	בקשות במקביל	בקשות/שנייה
Starter	512MB · 0.5 CPU	1	~40	~10
Standard	2GB · 1 CPU	4	~160	~40
Pro	4GB · 2 CPU	8	~320	~80
Pro Plus	8GB · 4 CPU	16	~640	~160

ההנחות: כ-400MB לעובד עם הספריות של OpenAI, Gemini ו-Supabase טעונות; 40 חוטים לעובד; כארבע שניות לקריאת מודל. המספרים מתקבלים מ-40 חוטים חלקי 4 שניות לכל עובד.

אבל אם מתקנים את הקוד

נתיב שמוגדר async ומחכה למודל במקום לחסום, מחזיר את החוט בזמן שהמודל חושב. אז מספר החוטים מפסיק להיות התקרה, והתקרה עוברת למגבלת הקצב של ספק המודל.



אחרי async, כל ארבע התוכניות מגיעות לאותו מספר, כי הגבול כבר אינו השרת אלא ספק המודל. שינוי קוד של יום עבודה שווה יותר מכל שדרוג חומרה שאפשר לקנות.

מה נשבר ראשון, לפי הסדר

1. הטיטור, בכ-40 משתמשים במקביל — בקשות נכנסות לתור וילד רואה מסך שנעצר.
2. כתיבות ל-Supabase, בכמה מאות שחקנים — כל תשובה כותבת שלוש שורות.
3. קריאות מ-Supabase, סביב 600 שחקנים — כפי שנמדד.
4. לא צפוי להישבר בשום סדר גודל שרלונטי כאן — Cloudflare

הארכיטקטורה שתסיר את התקרה

מה יש היום, ומה צריך כדי לא להיות מוגבל

היום: שני מסלולים נפרדים

מסלול הטיטור X

הדפדפן ← FastAPI (Render)
Render ← OpenAI / Gemini
רגיל: 40 חוטים חוסמים
תקרה: כ-40 במקביל

מסלול המשחקים ✓

הדפדפן ← Cloudflare (קבצים סטטיים)
הדפדפן ← Supabase (התקדמות)
לא נוגע בשרת כלל
תקרה: כ-600 במקביל

אותו אתר, שתי תקרות שרוחקות זו מזו פי חמישה עשר. כל עוד זה כך, "כמה משתמשים" היא שאלה בלי תשובה אחת — היא תלויה במה הילד עושה באותו רגע.

היעד: מה משנים, ומה זה נותן

3 · הגבלת קצב לכל משתמש

לשונית בלולאה לא
תיקח את כל השרת

2 · תור לעבודה ארוכה

יצירת שיעור ← תור
התשובה נאספת בהמשך

1 · נתיבים async

במקום חסימה await
החוט חוזר בזמן ההמתנה

6 · Supabase גדול יותר

הדבר היחיד שמזיז
את תקרת המשחקים

5 · יותר עובדים

רק אחרי async —
אחרת רק מכפילים זיכרון

4 · מטמון לתשובות חוזרות

שיעור שכבר נוצר
הקראה שכבר יוצרה

לא מוגבל, באמת

"בלי הגבלה" אינו קיים, אבל אפשר להעביר את הגבול למקום שקונים בכסף ולא בעבודה:

מה	איך	התקרה החדשה
קבצים סטטיים	כבר שם — Cloudflare	אינסופי למעשה
התקדמות ילדים	שדרוג תוכנית, Supabase	אלפים
קריאות מודל	תור + מטמון + async	מגבלת הספק
עומס פתאומי	תור מפריד בין קליטה לעיבוד	לא נופל, רק מאט

הסדר הנכון: הגבלת קצב היום, async אחריה, ותור רק כשבאמת צריך.
שלושת הראשונים הם קוד ואינם עולים כלום בחומרה. שלושת האחרונים הם כסף.
אין טעם לשלם על אף אחד מהאחרונים לפני שהראשונים נעשו.